ACM-ICPC Cheat Sheet

C++ – Quick reference for data structures and algorithms

1. Base Template

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define all(x) (x).begin(), (x).end()
typedef long long ll;
typedef pair<int,int> pii;
const double EPS = 1e-9;
const double PI = acos(-1);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // code here

    return 0;
}
```

Compile and run:

```
g++ -O2 -std=c++17 -Wall main.cpp -o main
./main < input.txt
```

Debug helper:

```
#ifdef DEBUG
#define SHOW(x) cerr << #x << " = " << (x) << "\n";
#else
#define SHOW(x)
#endif
```

2. STL Quick Reference

2.1 Sorting & comparators

```
sort(all(v));
sort(all(v), greater<int>());
// lambda
sort(all(v), [](int a, int b){ return a > b; });
// operator< as a member
struct Edge { int w;
  bool operator<(const Edge& o) const { return w < o.w; }
};
// functor for priority_queue (min-heap)
struct cmp { bool operator()(int a,int b){ return a > b; } };
priority_queue<int, vector<int>, cmp> pq;
priority_queue<int, vector<int>, greater<int>> pq2; // direct min-heap
```

2.2 Binary search

```
// require a sorted range
bool ok = binary_search(all(v), x);
auto it = lower_bound(all(v), x); // first >= x
```

```
auto it2 = upper_bound(all(v), x); // first > x
int idx = lower_bound(all(v), x) - v.begin();
```

2.3 Permutations

```
sort(all(v));
do {
    // process permutation v
} while (next_permutation(all(v)));
```

2.4 Key containers

```
vector<int> v(n, 0); // begin,end,size,push_back,pop_back
stack<int> st; // push,pop,top,empty
queue<int> q; // push,pop,front,empty
deque<int> dq; // push_front/back, pop_front/back
priority_queue<int> pq; // max-heap: push,pop,top
set<int> s; s.insert(x); // O(log n), unique, sorted
map<int,int> mp; mp[k] = v; // O(log n)
unordered_set<int> us; // amortized O(1)
unordered_map<int,int> um; // amortized O(1)
multiset<int> ms; // duplicates allowed
// map/set: find() -> end() if not found, count() -> 0/1
```

3. Data Structures

3.1 Union-Find (DSU)

```
struct DSU {
    vector<int> p, rnk;
    void init(int n){ p.resize(n); rnk.assign(n,0);
        iota(all(p), 0); }
    int find(int x){ return p[x]==x ? x : p[x]=find(p[x]); }
    bool unite(int a, int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(rnk[a]<rnk[b]) swap(a,b);
        p[b]=a;
        if(rnk[a]==rnk[b]) rnk[a]++;
        return true;
    }
};
```

3.2 Fenwick Tree (BIT)

[O(log n) update/query]

```
struct BIT {
    vector<ll> t; int n;
    void init(int n_){ n=n_; t.assign(n+1,0); }
    void update(int i, ll v){ // add v at position i (1-indexed)
        for(; i<=n; i+=i&(-i)) t[i]+=v;
    }
    ll query(int i){ // prefix sum [1..i]
        ll s=0;
        for(; i>0; i-=i&(-i)) s+=t[i];
        return s;
```

```
    }
    ll range(int l,int r){ return query(r)-query(l-1); }
};
// Range update + point query: use a BIT over differences
// add(l,r,v): update(l,v); update(r+1,-v); point(i) = query(i)
```

3.3 Segment Tree (lazy, range add + range sum)

[build O(n), query/update O(log n)]

```
struct SegTree {
    int n; vector<ll> sum, lazy;
    void init(int n_){ n=n_; sum.assign(4*n,0); lazy.assign(4*n,0); }
    void push(int nd,int l,int r){
        if(!lazy[nd]) return;
        int m=(l+r)/2;
        for(int c: {2*nd,2*nd+1}){
            int cl = (c==2*nd)? l : m+1, cr = (c==2*nd)? m : r;
            lazy[c]+=lazy[nd];
            sum[c]+=lazy[nd]*(cr-cl+1);
        }
        lazy[nd]=0;
    }
    void update(int nd,int l,int r,int ql,int qr,ll v){
        if(qr<l || r<ql) return;
        if(ql<=l && r<=qr){ sum[nd]+=v*(r-l+1); lazy[nd]+=v; return; }
        push(nd,l,r); int m=(l+r)/2;
        update(2*nd,l,m,ql,qr,v); update(2*nd+1,m+1,r,ql,qr,v);
        sum[nd]=sum[2*nd]+sum[2*nd+1];
    }
    ll query(int nd,int l,int r,int ql,int qr){
        if(qr<l || r<ql) return 0;
        if(ql<=l && r<=qr) return sum[nd];
        push(nd,l,r); int m=(l+r)/2;
        return query(2*nd,l,m,ql,qr)+query(2*nd+1,m+1,r,ql,qr);
    }
    // calls: update(1,0,n-1,l,r,v); query(1,0,n-1,l,r)
};
```

3.4 Sparse Table (static RMQ)

[build O(n log n), query O(1)]

```
struct SparseTable {
    vector<vector<int>> st; vector<int> lg;
    void init(vector<int>& a){
        int n=a.size();
        lg.assign(n+1,0);
        for(int i=2;i<=n;i++) lg[i]=lg[i/2]+1;
        int K=lg[n]+1;
        st.assign(K, vector<int>(n));
        st[0]=a;
        for(int j=1;j<K;j++)
            for(int i=0;i+(1<<j)<=n;i++)
                st[j][i]=min(st[j-1][i], st[j-1][i+(1<<(j-1))]);
    }
    int query(int l,int r){ // [l,r] inclusive, min
        int j=lg[r-l+1];
        return min(st[j][l], st[j][r-(1<<j)+1]);
    }
```

```
};
```

3.5 Trie

```
struct Trie {
    struct Node { int nxt[26]; bool end=false; };
    vector<Node> t;
    Trie(){ t.push_back(Node{}); memset(t[0].nxt,-1,sizeof(t[0].nxt)); }
    void insert(const string& s){
        int cur=0;
        for(char c: s){
            int x=c-'a';
            if(t[cur].nxt[x]==-1){
                t[cur].nxt[x]=t.size();
                t.push_back(Node{}); memset(t.back().nxt,-1,sizeof(t.back().nxt));
            }
            cur=t[cur].nxt[x];
        }
        t[cur].end=true;
    }
    bool search(const string& s){
        int cur=0;
        for(char c: s){
            int x=c-'a';
            if(t[cur].nxt[x]==-1) return false;
            cur=t[cur].nxt[x];
        }
        return t[cur].end;
    }
};
```

4. Advanced Trees

4.1 LCA (binary lifting)

[build O(n log n), query O(log n)]

```
const int LOG=20;
vector<int> g[MAXN];
int up[MAXN][LOG], depth[MAXN];
void dfs(int v,int p){
    up[v][0]=p;
    for(int j=1;j<LOG;j++)
        up[v][j]=up[up[v][j-1]][j-1];
    for(int to: g[v]) if(to!=p){
        depth[to]=depth[v]+1;
        dfs(to,v);
    }
}
int lca(int a,int b){
    if(depth[a]<depth[b]) swap(a,b);
    int diff=depth[a]-depth[b];
    for(int j=0;j<LOG;j++) if((diff>>j)&1) a=up[a][j];
    if(a==b) return a;
    for(int j=LOG-1;j>=0;j--)
        if(up[a][j]!=up[b][j]){ a=up[a][j]; b=up[b][j]; }
    return up[a][0];
}
// dist(a,b) = depth[a]+depth[b]-2*depth[lca(a,b)]
```

4.2 Heavy-Light Decomposition (skeleton)

[build O(n), query O(log^2 n)]

```
int par[MAXN], heavy[MAXN], head_[MAXN], pos_[MAXN], sz[MAXN], curpos;
int dfs_sz(int v,int p){
    sz[v]=1; par[v]=p; int mx=0, hv=-1;
    for(int to: g[v]) if(to!=p){
        int s=dfs_sz(to,v);
        sz[v]+=s;
        if(s>mx){ mx=s; hv=to; }
    }
    heavy[v]=hv; return sz[v];
}
void decompose(int v,int h){
    head_[v]=h; pos_[v]=curpos++;
    if(heavy[v]!=-1) decompose(heavy[v],h);
    for(int to: g[v]) if(to!=par[v] && to!=heavy[v]) decompose(to,to);
}
// query(a,b): while head_[a]!=head_[b], jump the deeper one up to
// head_[..], query the segment tree using pos_[]
```

4.3 Centroid Decomposition (skeleton)

[build O(n log n)]

```
bool removed[MAXN]; int subsz[MAXN];
int calcSize(int v,int p){
    subsz[v]=1;
    for(int to: g[v]) if(to!=p && !removed[to])
        subsz[v]+=calcSize(to,v);
    return subsz[v];
}
int findCentroid(int v,int p,int treeSz){
    for(int to: g[v]) if(to!=p && !removed[to])
        if(subsz[to]>treeSz/2) return findCentroid(to,v,treeSz);
    return v;
}
void decompose(int v){
    int treeSz=calcSize(v,-1);
    int c=findCentroid(v,-1,treeSz);
    removed[c]=true;
    // process centroid c here
    for(int to: g[c]) if(!removed[to]) decompose(to);
}
```

5. Strings

5.1 KMP

[O(n+m)]

```
vector<int> prefixFn(const string& s){
    int n=s.size();
    vector<int> pi(n,0);
    for(int i=1;i<n;i++){
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
```

```
        pi[i]=j;
    }
    return pi;
}
vector<int> kmpSearch(const string& text, const string& pat){
    string s = pat+"#"+text;
    vector<int> pi = prefixFn(s), res;
    for(int i=(int)pat.size()+1;i<(int)s.size();i++)
        if(pi[i]==(int)pat.size())
            res.push_back(i-2*(int)pat.size());
    return res; // 0-indexed positions where pat occurs in text
}
```

5.2 Manacher (longest palindromic substring)

[O(n)]

```
string longestPalindrome(const string& str){
    string s="^#";
    for(char c: str){ s+=c; s+="#"; }
    s+="$";
    int n=s.size();
    vector<int> p(n,0);
    int c=0, r=0;
    for(int i=1;i<n-1;i++){
        if(i<r) p[i]=min(r-i, p[2*c-i]);
        while(s[i+p[i]+1]==s[i-p[i]-1]) p[i]++;
        if(i+p[i]>r){ c=i; r=i+p[i]; }
    }
    int best=0, cen=0;
    for(int i=1;i<n-1;i++) if(p[i]>best){ best=p[i]; cen=i; }
    int start=(cen-best)/2;
    return str.substr(start, best);
}
```

5.3 Z-function (bonus)

[O(n)]

```
vector<int> zFunction(const string& s){
    int n=s.size();
    vector<int> z(n,0);
    int l=0,r=0;
    for(int i=1;i<n;i++){
        if(i<r) z[i]=min(r-i, z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]>r){ l=i; r=i+z[i]; }
    }
    return z;
}
```

6. Graph

6.1 Representation + BFS/DFS

```
vector<int> g[MAXN]; bool vis[MAXN];
void addEdge(int u,int v){ g[u].pb(v); g[v].pb(u); } // undirected
```

```
void bfs(int s){
    queue<int> q; q.push(s); vis[s]=true;
    while(!q.empty()){
        int cur=q.front(); q.pop();
        for(int to: g[cur]) if(!vis[to]){
            vis[to]=true; q.push(to);
        }
    }
}
void dfs(int v){
    vis[v]=true;
    for(int to: g[v]) if(!vis[to]) dfs(to);
}
```

6.2 Topological sort (Kahn)

[O(V+E)]

```
vector<int> topoSort(int n, vector<int> adj[]){
    vector<int> indeg(n,0), order;
    for(int u=0;u<n;u++) for(int v: adj[u]) indeg[v]++;
    queue<int> q;
    for(int i=0;i<n;i++) if(!indeg[i]) q.push(i);
    while(!q.empty()){
        int u=q.front(); q.pop(); order.pb(u);
        for(int v: adj[u]) if(--indeg[v]==0) q.push(v);
    }
    return order; // size < n if there is a cycle
}
```

6.3 Dijkstra

[O((V+E) log V), non-negative weights]

```
vector<pii> g[MAXN]; // (neighbor, weight)
vector<ll> dist_(MAXN, LLONG_MAX);
void dijkstra(int s){
    priority_queue<pii, vector<pii>, greater<pii>> pq; // (dist, node)
    dist_[s]=0; pq.push({0,s});
    while(!pq.empty()){
        auto [d,u]=pq.top(); pq.pop();
        if(d>dist_[u]) continue;
        for(auto [v,w]: g[u]) if(dist_[u]+w<dist_[v]){
            dist_[v]=dist_[u]+w;
            pq.push({dist_[v], v});
        }
    }
}
```

6.4 Bellman-Ford

[O(V*E), detects negative cycles]

```
struct Edge{ int u,v,w; };
vector<Edge> edges;
bool bellmanFord(int n,int s, vector<ll>& dist_){
    dist_.assign(n, LLONG_MAX); dist_[s]=0;
    for(int i=0;i<n-1;i++)
```

```
        for(auto& e: edges)
            if(dist_[e.u]!=LLONG_MAX && dist_[e.u]+e.w<dist_[e.v])
                dist_[e.v]=dist_[e.u]+e.w;
    for(auto& e: edges) // check for negative cycle
        if(dist_[e.u]!=LLONG_MAX && dist_[e.u]+e.w<dist_[e.v])
            return false;
    return true;
}
```

6.5 Floyd-Warshall

[O(V^3), all pairs]

```
ll d[MAXV][MAXV]; // init: d[i][i]=0, d[i][j]=weight or INF
void floydWarshall(int n){
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(d[i][k]+d[k][j]<d[i][j])
                    d[i][j]=d[i][k]+d[k][j];
}
```

6.6 SPFA (queue-based Bellman-Ford)

```
bool inq[MAXN]; int cnt[MAXN]; // cnt to detect negative cycle
bool spfa(int n,int s, vector<ll>& dist_){
    dist_.assign(n, LLONG_MAX); dist_[s]=0;
    queue<int> q; q.push(s); inq[s]=true;
    while(!q.empty()){
        int u=q.front(); q.pop(); inq[u]=false;
        for(auto [v,w]: g[u]) if(dist_[u]+w<dist_[v]){
            dist_[v]=dist_[u]+w;
            if(!inq[v]){
                q.push(v); inq[v]=true;
                if(++cnt[v]>n) return false; // negative cycle
            }
        }
    }
    return true;
}
```

6.7 MST: Kruskal

[O(E log E)]

```
struct Edge{ int u,v,w; };
ll kruskal(int n, vector<Edge>& edges){
    sort(all(edges), [](Edge&a,Edge&b){return a.w<b.w;});
    DSU dsu; dsu.init(n);
    ll cost=0; int used=0;
    for(auto& e: edges)
        if(dsu.unite(e.u,e.v)){ cost+=e.w; used++; }
    return (used==n-1)? cost : -1; // -1 if not connected
}
```

6.8 MST: Prim

[O(E log V)]

```
ll prim(int n, vector<pii> g[]){
    vector<bool> in(n,false);
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0,0}); ll cost=0; int cnt=0;
    while(!pq.empty() && cnt<n){
        auto [w,u]=pq.top(); pq.pop();
        if(in[u]) continue;
        in[u]=true; cost+=w; cnt++;
        for(auto [v,wt]: g[u]) if(!in[v]) pq.push({wt,v});
    }
    return (cnt==n)? cost : -1;
}
```

6.9 Bipartite matching (Kuhn)

[O(V*E)]

```
vector<int> adj[MAXN]; int matchR[MAXN]; bool used[MAXN];
bool tryKuhn(int v){
    for(int to: adj[v]){
        if(used[to]) continue;
        used[to]=true;
        if(matchR[to]==-1 || tryKuhn(matchR[to])){
            matchR[to]=v; return true;
        }
    }
    return false;
}
int maxMatching(int nLeft, int nRight){
    fill(matchR, matchR+nRight, -1);
    int res=0;
    for(int v=0; v<nLeft; v++){
        fill(used, used+nRight, false);
        if(tryKuhn(v)) res++;
    }
    return res;
}
```

6.10 Max Flow (Dinic)

[O(V^2E), O(E sqrt(V)) on bipartite graphs]

```
struct Dinic {
    struct E{ int to,cap,flow; };
    vector<E> edges; vector<int> g[MAXN];
    int lvl[MAXN], it[MAXN], n,s,t;
    void init(int n_){ n=n_; edges.clear(); for(int i=0;i<n;i++) g[i].clear(); }
    void addEdge(int u,int v,int cap){
        g[u].pb(edges.size()); edges.pb({v,cap,0});
        g[v].pb(edges.size()); edges.pb({u,0,0}); // reverse edge
    }
    bool bfs(){
        fill(lvl, lvl+n, -1); lvl[s]=0;
        queue<int> q; q.push(s);
        while(!q.empty()){
            int u=q.front(); q.pop();
```

```
            for(int id: g[u]){
                auto&e=edges[id];
                if(lvl[e.to]<0 && e.cap>e.flow){
                    lvl[e.to]=lvl[u]+1; q.push(e.to);
                }
            }
        }
        return lvl[t]>=0;
    }
    int dfs(int u,int pushed){
        if(u==t || !pushed) return pushed;
        for(int& i=it[u]; i<(int)g[u].size(); i++){
            int id=g[u][i]; auto&e=edges[id];
            if(lvl[u]+1!=lvl[e.to] || e.cap<=e.flow) continue;
            int d=dfs(e.to, min(pushed, e.cap-e.flow));
            if(d>0){ e.flow+=d; edges[id^1].flow-=d; return d; }
        }
        return 0;
    }
    int maxflow(int s_,int t_){
        s=s_; t=t_; int flow=0;
        while(bfs()){
            fill(it, it+n, 0);
            while(int pushed=dfs(s, INT_MAX)) flow+=pushed;
        }
        return flow;
    }
};
```

6.11 Bridges & articulation points (Tarjan)

[O(V+E)]

```
int disc[MAXN], low[MAXN], timer_=0; bool isArt[MAXN];
vector<pii> bridges;
void dfsBridge(int u,int p){
    disc[u]=low[u]=++timer_; int children=0;
    for(int v: g[u]){
        if(v==p) continue;
        if(disc[v]){ low[u]=min(low[u], disc[v]); continue; }
        children++;
        dfsBridge(v,u);
        low[u]=min(low[u], low[v]);
        if(low[v]>disc[u]) bridges.pb({u,v}); // bridge u-v
        if(p!=-1 && low[v]>=disc[u]) isArt[u]=true; // articulation point
    }
    if(p==-1 && children>1) isArt[u]=true; // root case
}
```

6.12 Strongly Connected Components (Tarjan)

[O(V+E), directed graph]

```
int disc2[MAXN], low2[MAXN], timer2=0, sccId[MAXN], sccCnt=0;
bool onStack[MAXN]; stack<int> stck;
void sccDfs(int u){
    disc2[u]=low2[u]=++timer2;
    stck.push(u); onStack[u]=true;
    for(int v: g[u]){
        if(!disc2[v]){ sccDfs(v); low2[u]=min(low2[u],low2[v]); }
```

```
        else if(onStack[v]) low2[u]=min(low2[u], disc2[v]);
    }
    if(low2[u]==disc2[u]){
        while(true){
            int v=stck.top(); stck.pop(); onStack[v]=false;
            sccId[v]=sccCnt;
            if(v==u) break;
        }
        sccCnt++;
    }
}
```

7. Number Theory & Math

7.1 GCD / LCM / Extended Euclid

```
ll gcd(ll a, ll b){ return b? gcd(b, a%b) : a; }
ll lcm(ll a, ll b){ return a/gcd(a,b)*b; }
// a*x + b*y = gcd(a,b)
ll extGcd(ll a, ll b, ll& x, ll& y){
    if(!b){ x=1; y=0; return a; }
    ll x1,y1;
    ll d=extGcd(b, a%b, x1, y1);
    x=y1; y=x1 - (a/b)*y1;
    return d;
}
// modular inverse of a under modulus m (requires gcd(a,m)=1)
ll modInverse(ll a, ll m){
    ll x,y; extGcd(a,m,x,y);
    return ((x%m)+m)%m;
}
```

7.2 Fast exponentiation

[O(log n)]

```
ll power(ll base, ll exp, ll mod){
    ll res=1; base%=mod;
    while(exp>0){
        if(exp&1) res=res*base%mod;
        base=base*base%mod;
        exp>>=1;
    }
    return res;
}
```

7.3 Sieve of Eratosthenes

[O(n log log n)]

```
vector<bool> isComposite;
vector<int> primes;
void sieve(int n){
    isComposite.assign(n+1, false);
    for(int i=2;i<=n;i++){
        if(!isComposite[i]) primes.pb(i);
        for(int p: primes){
            if((ll)i*p>n) break;
```

```
            isComposite[i*p]=true;
            if(i%p==0) break;
        }
    }
}
```

7.4 Primality & factorization

```
bool isPrime(ll n){
    if(n<2) return false;
    for(ll i=2;i*i<=n;i++) if(n%i==0) return false;
    return true;
}
vector<ll> factorize(ll n){
    vector<ll> f;
    for(ll d=2; d*d<=n; d++)
        while(n%d==0){ f.pb(d); n/=d; }
    if(n>1) f.pb(n);
    return f;
}
// Miller-Rabin (deterministic for n < 3.3*10^14 with these bases)
bool millerRabin(ll n){
    if(n<4) return n==2||n==3;
    ll d=n-1; int r=0;
    while(d%2==0){ d/=2; r++; }
    for(ll a: {2,3,5,7,11,13,17,19,23,29,31,37}){
        if(n==a) return true;
        if(n%a==0) return false;
        ll x=power(a,d,n);
        if(x==1||x==n-1) continue;
        bool comp=true;
        for(int i=0;i<r-1;i++){
            x=(__int128)x*x%n;
            if(x==n-1){ comp=false; break; }
        }
        if(comp) return false;
    }
    return true;
}
```

7.5 Combinatorics mod p (precomputed factorials)

```
const int MOD=1e9+7, NMAX=2e5+5;
ll fact[NMAX], inv_fact[NMAX];
void precomputeFact(){
    fact[0]=1;
    for(int i=1;i<NMAX;i++) fact[i]=fact[i-1]*i%MOD;
    inv_fact[NMAX-1]=power(fact[NMAX-1], MOD-2, MOD);
    for(int i=NMAX-2;i>=0;i--) inv_fact[i]=inv_fact[i+1]*(i+1)%MOD;
}
ll nCr(int n,int r){
    if(r<0||r>n) return 0;
    return fact[n]*inv_fact[r]%MOD*inv_fact[n-r]%MOD;
}
```

7.6 Game theory: Nim & Sprague-Grundy

- **Nim**: with piles $n_1, \ldots, n_k$, the position is losing (for the player about to move) $\iff n_1 \oplus n_2 \oplus \cdots \oplus n_k = 0$.
- **Sprague-Grundy**: $g(\text{state}) = \text{mex}\{g(s') : s' \text{ is a successor state}\}$; mex = smallest non-negative

integer not present in the set. XOR of the grundy numbers of independent subgames gives the grundy number of the compound game; grundy 0 = losing position.

```
int mex(vector<int>& s){
    vector<bool> seen(s.size()+1, false);
    for(int x: s) if(x<=(int)s.size()) seen[x]=true;
    int m=0; while(m<(int)seen.size() && seen[m]) m++;
    return m;
}
```

7.7 Big integers

For 10^{18}–10^{38} use `__int128` (no direct I/O, you must write manual input/output routines). For arbitrary precision, implement a bigint with `vector<int>` in base 10^4 (schoolbook add/sub/mul, division by a small integer).

8. Geometry

8.1 Point & basic operations

```
struct Point {
    double x, y;
    Point operator-(const Point& o) const { return {x-o.x, y-o.y}; }
    Point operator+(const Point& o) const { return {x+o.x, y+o.y}; }
    double cross(const Point& o) const { return x*o.y - y*o.x; }
    double dot(const Point& o) const { return x*o.x + y*o.y; }
    double norm() const { return sqrt(x*x+y*y); }
};
double dist(Point a, Point b){ return (a-b).norm(); }
// orientation of a,b,c: >0 ccw turn, <0 cw turn, 0 collinear
double orientation(Point a, Point b, Point c){
    return (b-a).cross(c-a);
}
// distance from point p to segment a-b
double distPointSegment(Point p, Point a, Point b){
    Point ab=b-a, ap=p-a;
    double t = ab.dot(ap) / ab.dot(ab);
    t = max(0.0, min(1.0, t));
    Point proj = a + Point{ab.x*t, ab.y*t};
    return dist(p, proj);
}
```

8.2 Convex Hull (monotone chain / Andrew)

[O(n log n)]

```
vector<Point> convexHull(vector<Point> pts){
    int n=pts.size(), k=0;
    if(n<3) return pts;
    sort(pts.begin(), pts.end(), [](Point&a, Point&b){
        return a.x<b.x || (a.x==b.x && a.y<b.y);
    });
    vector<Point> hull(2*n);
    // lower hull
    for(int i=0;i<n;i++){
        while(k>=2 && orientation(hull[k-2],hull[k-1],pts[i])<=0) k--;
        hull[k++]=pts[i];
    }
    // upper hull
```

```
    for(int i=n-2, t=k+1; i>=0; i--){
        while(k>=t && orientation(hull[k-2],hull[k-1],pts[i])<=0) k--;
        hull[k++]=pts[i];
    }
    hull.resize(k-1); // last point == first point
    return hull;
}
```

Notes: cross product $a \times b > 0$ means b is to the left of a; $a \times b$ is the signed area of the parallelogram (divide by 2 for the triangle area).

9. Tricks & Miscellaneous

9.1 Bit manipulation

```
#define GET_BIT(n,i) (((n)>>(i)) & 1)
#define SET_BIT(n,i) ((n) | (1LL<<(i)))
#define CLR_BIT(n,i) ((n) & ~(1LL<<(i)))
// __builtin_popcount(x) -> number of 1 bits (int)
// __builtin_popcountll(x) -> long long version
// __builtin_clz(x) / __builtin_ctz(x) -> leading/trailing zeros
// x & (x-1) -> turns off the lowest set bit
// x & (-x) -> isolates the lowest set bit
```

9.2 Integer square root

```
ll isqrt(ll a){
    ll l=0, r=2e9;
    while(l<r){
        ll m=l+(r-l+1)/2;
        if(m*m<=a) l=m; else r=m-1;
    }
    return l;
}
```

9.3 LIS – Longest Increasing Subsequence

[O(n log n)]

```
int lis(vector<int>& a){
    vector<int> tail; // tail[i] = smallest tail value of an LIS of size i+1
    for(int x: a){
        auto it = lower_bound(all(tail), x); // use upper_bound for non-decreasing
        if(it==tail.end()) tail.pb(x);
        else *it = x;
    }
    return tail.size();
}
```

9.4 Generic binary search on the answer

```
// find the smallest x such that pred(x) is true (pred is monotonic)
ll bsearch(ll lo, ll hi, function<bool(ll)> pred){
    while(lo<hi){
        ll mid=lo+(hi-lo)/2;
        if(pred(mid)) hi=mid; else lo=mid+1;
    }
    return lo;
```

```
}
```

9.5 Generate all subsets

```
int n=v.size();
for(int mask=0; mask<(1<<n); mask++){
    vector<int> subset;
    for(int i=0;i<n;i++)
        if(mask & (1<<i)) subset.pb(v[i]);
    // process subset
}
```

9.6 Passing 2D arrays to functions

```
int arr[10][10];
void f(int a[][10]){ /* ... */ } // fixed number of columns
f(arr);

vector<vector<int>> arr2(10, vector<int>(10));
void f2(vector<vector<int>>& a){ /* ... */ } // recommended
f2(arr2);
```

10. Extra Structures & Algorithms

10.1 Suffix Array (O(n log n) build)

[build O(n log n), LCP array O(n)]

```
struct SuffixArray {
    string s; int n;
    vector<int> sa, rank_, tmp, lcp;
    void build(const string& str){
        s = str + "\1"; n = s.size();
        sa.resize(n); rank_.resize(n); tmp.resize(n);
        iota(all(sa), 0);
        for(int i=0;i<n;i++) rank_[i]=s[i];
        for(int k=1;k<n;k<<=1){
            auto cmp=[&](int a,int b){
                if(rank_[a]!=rank_[b]) return rank_[a]<rank_[b];
                int ra = a+k<n ? rank_[a+k] : -1;
                int rb = b+k<n ? rank_[b+k] : -1;
                return ra<rb;
            };
            sort(all(sa), cmp);
            tmp[sa[0]]=0;
            for(int i=1;i<n;i++)
                tmp[sa[i]] = tmp[sa[i-1]] + (cmp(sa[i-1],sa[i]) ? 1 : 0);
            rank_=tmp;
            if(rank_[sa[n-1]]==n-1) break;
        }
    }
    void buildLCP(){ // Kasai's algorithm, O(n)
        vector<int> rk(n);
        for(int i=0;i<n;i++) rk[sa[i]]=i;
        lcp.assign(n,0);
        int h=0;
        for(int i=0;i<n;i++){
            if(rk[i]>0){
                int j=sa[rk[i]-1];
```

```
            while(i+h<n && j+h<n && s[i+h]==s[j+h]) h++;
            lcp[rk[i]]=h;
            if(h>0) h--;
        } else h=0;
    }
  }
};
```

10.2 Matrix exponentiation (linear recurrences)

[O(k^3 log n) for a k x k matrix]

```
typedef vector<vector<ll>> Matrix;
const ll MOD2 = 1e9+7;
Matrix matMul(const Matrix& a, const Matrix& b){
    int n=a.size(), m=b[0].size(), k=b.size();
    Matrix c(n, vector<ll>(m, 0));
    for(int i=0;i<n;i++)
        for(int j=0;j<k;j++){
            if(!a[i][j]) continue;
            for(int l=0;l<m;l++)
                c[i][l] = (c[i][l] + a[i][j]*b[j][l]) % MOD2;
        }
    return c;
}
Matrix matPow(Matrix a, ll p){
    int n=a.size();
    Matrix res(n, vector<ll>(n,0));
    for(int i=0;i<n;i++) res[i][i]=1; // identity
    while(p>0){
        if(p&1) res = matMul(res, a);
        a = matMul(a, a);
        p >>= 1;
    }
    return res;
}
// Fibonacci example: [[1,1],[1,0]]^n gives F(n+1), F(n)
```

10.3 FFT (polynomial multiplication)

[O(n log n)]

```
typedef complex<double> cd;
void fft(vector<cd>& a, bool invert){
    int n = a.size();
    if(n==1) return;
    for(int i=1, j=0; i<n; i++){
        int bit = n>>1;
        for(; j&bit; bit>>=1) j ^= bit;
        j ^= bit;
        if(i<j) swap(a[i], a[j]);
    }
    for(int len=2; len<=n; len<<=1){
        double ang = 2*PI/len * (invert? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for(int i=0; i<n; i+=len){
            cd w(1);
            for(int j=0; j<len/2; j++){
                cd u = a[i+j], v = a[i+j+len/2]*w;
                a[i+j] = u+v; a[i+j+len/2] = u-v;
```

```
                w *= wlen;
            }
        }
    }
    if(invert) for(cd& x: a) x /= n;
}
vector<ll> multiply(vector<ll> const& a, vector<ll> const& b){
    vector<cd> fa(all(a)), fb(all(b));
    int n=1;
    while(n < (int)(a.size()+b.size())) n <<= 1;
    fa.resize(n); fb.resize(n);
    fft(fa,false); fft(fb,false);
    for(int i=0;i<n;i++) fa[i] *= fb[i];
    fft(fa,true);
    vector<ll> res(n);
    for(int i=0;i<n;i++) res[i] = round(fa[i].real());
    return res; // remember to propagate carries afterwards
}
```

10.4 Range Minimum Query with a Segment Tree (no lazy)

[build O(n), query O(log n)]

```
struct MinSegTree {
    int n; vector<int> t;
    void init(int n_){ n=n_; t.assign(4*n, INT_MAX); }
    void update(int nd,int l,int r,int pos,int val){
        if(l==r){ t[nd]=val; return; }
        int m=(l+r)/2;
        if(pos<=m) update(2*nd,l,m,pos,val);
        else update(2*nd+1,m+1,r,pos,val);
        t[nd] = min(t[2*nd], t[2*nd+1]);
    }
    int query(int nd,int l,int r,int ql,int qr){
        if(qr<l || r<ql) return INT_MAX;
        if(ql<=l && r<=qr) return t[nd];
        int m=(l+r)/2;
        return min(query(2*nd,l,m,ql,qr), query(2*nd+1,m+1,r,ql,qr));
    }
};
```

10.5 Advanced matching (note)

For maximum matching on general (non-bipartite) graphs, use **Edmonds' Blossom Algorithm** ($O(V^3)$). It is rarely needed at university-level contests; if it appears, look up a tested library implementation rather than writing it from scratch under time pressure.

11. Complexity Cheat Sheet

Algorithm / Structure	**Complexity**
Sort (std::sort)	$O(n \log n)$
Binary search	$O(\log n)$
DSU (union by rank + path compression)	$O(\alpha(n))$ amortized
Fenwick Tree / BIT	$O(\log n)$
Segment Tree	$O(\log n)$
Sparse Table	$O(1)$ query, $O(n \log n)$ build
BFS / DFS	$O(V + E)$
Dijkstra (heap)	$O((V + E) \log V)$
Bellman-Ford	$O(VE)$
Floyd-Warshall	$O(V^3)$
Kruskal / Prim (MST)	$O(E \log E)$ / $O(E \log V)$
Dinic (max flow)	$O(V^2 E)$
Kuhn (bipartite matching)	$O(VE)$
KMP / Z-function	$O(n + m)$
Manacher	$O(n)$
Suffix Array (doubling)	$O(n \log n)$
FFT	$O(n \log n)$
Fast exponentiation	$O(\log n)$
Sieve of Eratosthenes	$O(n \log \log n)$
Miller-Rabin	$O(k \log^3 n)$
Convex Hull	$O(n \log n)$

Rough operations-per-second budget: with a 1–2 second time limit, aim for about 10^8–10^9 simple operations. Rule of thumb for choosing an approach given n:

- $n \leq 10$: brute force / exponential ($O(2^n)$, $O(n!)$)
- $n \leq 20$: bitmask DP ($O(2^n \cdot n)$)
- $n \leq 500$: $O(n^3)$
- $n \leq 5000$: $O(n^2)$
- $n \leq 10^6$: $O(n \log n)$ or $O(n)$
- $n \leq 10^8$: $O(n)$ with very light constant, or $O(\log n)$

Condensed cheat sheet built from a personal ICPC reference README – for contest use.